

AI Model Accuracy Report

Three LoRA adapters on Llama 3.1 8B Instruct and the XGBoost deal scorer — measured on held-out data, through the live servers, and after fixing the defects the evaluation uncovered.

Evaluation date	17 September 2026
Repository	D:\CRM · branch backup-restore-docs (fixes uncommitted at time of writing)
Hardware	NVIDIA RTX A4000, 16 GB · Windows 11 · Python 3.14.6
Language model	Llama 3.1 8B Instruct, 4-bit NF4 · LoRA r=16, α=32 · greedy decoding
Deal scorer	XGBoost 3.3.0 regressor · bundle deal_score_v1.0.0_20260802_062230Z
Held-out data	Seeded 90/10 split (seed 42), identical to each adapter's training run

1. Executive summary

All three adapters were evaluated on the business task they perform, not on token accuracy. Every adapter produced valid, scoreable output on 100% of held-out rows. The weakest link in the deal-scoring chain is now the XGBoost regressor, not the language model.

Model	Task	Held-out rows	Headline result	Verdict
lead_scoring adapter	Lead profile → score, Hot/Warm/Cold	50	100% served (with post-processing) · 90% raw model qualification · 58% raw exact score	Production-ready
lead_meeting adapter	Meeting notes → 5 signals	50	74% all 5 fields · 84% priority band · score MAE 2.8 · every field far above baseline	Good; sentiment weakest
deal_state adapter	State + notes → 17 fields	80	Categoricals 100% · end-to-end deal-score MAE 0.33 pts · 100% same band	Production-ready
XGBoost deal scorer	17 fields → deal score 0–100	300 / 200 / 200	MAE 4.6 on its own test set · MAE 11.9–13.2 on two other labelled datasets	Generalisation risk

Most important finding. The deal scorer reproduces its reported accuracy (MAE 4.6, R^2 0.89) only on its own test set. On the two 200-row datasets committed to the repository it is off by 12–13 points with a 6–8 point systematic bias, and assigns the right priority band only 62–68% of the time. Which dataset carries the true labels must be resolved before the scores are relied on.

Second finding. Lead scoring's 100% accuracy is delivered by deterministic post-processing (*reconcile_output*), not by the model alone. The raw model gets 94.4% of individual factor points right but its stated total disagreed with its own bullets in 42% of replies. The post-processing must never be removed.

2. How accuracy was measured

- **Held-out rows only.** Each evaluation rebuilds the exact 10% split its adapter's training run held out (same filter, seed 42, shuffle). Training logs confirm no rows were dropped, so the splits match.
- **Task-level metrics.** Token accuracy is inflated by JSON scaffolding (the deal_state adapter scores 99.6% token accuracy but 31% whole-state accuracy), so field accuracy, band agreement and score error are reported instead.
- **Production parity.** Prompts rendered by training, evaluation and the live server were compared byte-for-byte: 200/200 identical for deal_state and 494/494 for lead_meeting. The meeting adapter was also re-run through

the live HTTP route after the decoding fix and matched the offline evaluation exactly.

- **End-to-end.** deal_state output was scored by the real XGBoost bundle in strict mode and compared with the score of the ground-truth state, so extraction errors are expressed in deal-score points.

3. lead_scoring adapter

Reads a lead profile and writes a score out of 100 as five factor bullets (up to 20 points each), a Hot/Warm/Cold qualification, a priority and a recommended action. Evaluated on 50 held-out leads.

Metric	Raw model output	Bullet-sum correction only	Full reconcile_output (served)
Format valid	50/50 · 100%	50/50 · 100%	50/50 · 100%
Qualification correct	45/50 · 90%	49/50 · 98%	50/50 · 100%
Priority correct	45/50 · 90%	49/50 · 98%	50/50 · 100%
Score exactly right	29/50 · 58%	36/50 · 72%	50/50 · 100%
Score within ±5	46/50 · 92%	50/50 · 100%	50/50 · 100%
Score within ±10	50/50 · 100%	50/50 · 100%	50/50 · 100%
Mean absolute score error	2.50 pts	1.40 pts	0.00 pts

- Per-factor point accuracy of the raw model: **236/250 (94.4%)**. The model mostly judges each factor correctly; its errors are in adding the five numbers up.
- Header score disagreed with its own bullet total in **21/50 replies (42%)**; all were corrected by *reconcile_output*, which recomputes four factors from fixed range rules and the fifth when the requirement text matches a known template.
- Live check through the CRM route (5 reference leads): 5/5 exact score and label.
- Previously only token accuracy (96.1%) was known for this adapter; this is its first task-level measurement.

4. lead_meeting adapter

Reads free-text qualification-meeting notes and extracts five signals as JSON. Evaluated on 50 held-out meetings; the baseline is what a model ignoring the notes would score by always answering the most common value.

Overall metric	Result
Valid JSON / needed repair	50/50 (100%) · 0 repairs
All 5 fields correct	37/50 · 74%
Priority band match	42/50 · 84%
Meeting score mean absolute error	2.80 points out of 100

Field	Accuracy	Always-majority baseline	Most common mistakes
customer_sentiment	82%	42% (Neutral)	Neutral→Positive ×4, Positive→Neutral ×2, Neutral→Negative ×1
buying_intent	100%	44% (High)	—
decision_maker_involvement	96%	46% (Present)	Indirect→Present ×1, Indirect→Absent ×1
customer_urgency	100%	46% (Medium)	—
product_interest_level	96%	38% (High)	Low→Medium ×1, Medium→Low ×1

Every field is 40–58 points above its baseline, so the adapter reads the notes rather than the label distribution. Sentiment is the only field below 96%, and its errors lean towards reading neutral meetings as positive.

5. deal_state adapter

Takes the previous 17-field deal state plus the latest meeting notes and returns the updated state that the XGBoost scorer consumes. Evaluated on all 80 held-out rows (the earlier figures used 30).

Metric	17 Sep 2026 (80 rows)	Earlier provenance (30 rows)
Valid JSON	80/80 · 100%	100%
Accepted by scorer with zero repairs	80/80 · 100%	100%
All 17 fields exactly right	25/80 · 31.2%	33.3%
Mean field accuracy	94.5%	94.9%
Changed recall — fields that had to move and did	376/443 · 84.9%	86.6%
Unchanged precision — fields that had to stay and did	909/917 · 99.1%	98.6%

Per-field accuracy

Field(s)	Accuracy	Note
12 categorical fields + total_meetings	100%	Every word-valued field correct on all 80 rows
relationship_strength	91.2%	Numeric 0–10 judgement
engagement_score	76.2%	Weighted formula the model must compute exactly
lead_score	38.8%	Multi-term weighted sum; an 8B model does not do this arithmetic reliably

End-to-end: what extraction errors cost in deal-score points

Measure	17 Sep 2026 (80 rows)	Earlier (30 rows)
Mean absolute error	0.33 pts	0.29 pts
Median absolute error	0.13 pts	0.14 pts
Worst case	1.91 pts	1.43 pts
Bias (mean signed error)	−0.02 pts	+0.07 pts
Within 1 / 3 / 5 points	88.8% / 100% / 100%	— / 100% / —
Same Low / Medium / High band	80/80 · 100%	100%

The weak numeric fields barely move the deal score because the regressor leans on the categoricals, which are 100% correct — and the backend already recomputes lead_score and engagement_score before scoring. Extraction adds 0.33 points of error to a regressor whose own test error is 4.4 points.

Live chain test (5 consecutive meetings through :8001 → :8000): 5/5 states accepted, 0 repairs, score trajectory 46 → 53 → 78 → 67 → 80.

6. XGBoost deal scorer

Gradient-boosted regressor (max_depth 2, 1,500 trees, learning rate 0.03) trained on 1,000 rows. The saved model file used by the live server is byte-identical to the newest bundle in xgboost/models.

Reported training metrics (from the bundle)

Metric	Test (200 rows)	Train (800 rows)	5-fold CV
R ²	0.9026	0.9536	0.9048 ± 0.0142
MAE (points)	4.357	2.972	—
RMSE (points)	5.554	3.781	5.382 ± 0.287
MAPE	10.26%	6.97%	—

Independent re-scoring against labelled datasets in the repository

Dataset	n	MAE	RMSE	R ²	Bias	Within ±5 / ±10	Priority band	Worst
outputs/scored_predictions.csv	300	4.63	5.81	0.890	−0.05	60.7% / 91.0%	83.3%	15.3
deal_score_dataset_200.csv	200	11.94	14.42	0.834	+7.75	24.0% / 45.5%	68.0%	34.9
outputs/deal_score_testing_200_OUTPUT.csv	200	13.19	15.95	0.776	−5.88	21.5% / 42.0%	61.5%	41.3

Priority band uses the service's own cut-offs (≥75 HIGH, ≥50 MEDIUM, ≥25 LOW, otherwise VERY LOW). Unit tests: 33/33 pass.

Interpretation. The model matches its reported quality only on data from its own generator. The two 200-row files are biased in opposite directions (+7.75 and −5.88), which points to different labelling rules rather than random noise. Either those labels are not ground truth, or the model does not generalise beyond its training distribution. The 1,000-row training CSV is not in the repository (its provenance records a path in a personal Downloads folder), so this cannot yet be settled.

7. Defects found and fixed

The evaluation surfaced the following problems. All were fixed and verified on 17 September 2026; the changes are uncommitted on branch backup-restore-docs.

#	Problem	Effect	Fix and verification
1	AI venv lacked trl, datasets, pandas, xgboost, joblib, matplotlib; requirements-train.txt pinned an outdated xgboost 3.2.0 with an incorrect note	All three evaluation scripts crashed on import; 29 contract tests silently skipped	Dependencies installed and pins corrected (xgboost 3.3.0, matplotlib added). AI tests: 145 pass, 0 skipped
2	serve_api.py /health read metrics from the wrong key	test_r2 always null	Reads bundle metrics; live /health now returns 0.9026. New regression test (fails before, passes after)
3	xgboost/requirements.txt omitted fastapi, uvicorn, pydantic and listed matplotlib as training-only	A clean install of the serving set could not start the API	Serving dependencies added and pinned
4	DealScoringClient.score(Deal) sent "" for 8 empty categorical fields	Strict scorer returns 400; the deal is silently left unscored	Falls back to DealParameters.DEFAULTS. New DealScoringClientTest: 3/3 pass
5	Meeting route served at repetition penalty 1.05; evaluation measured 1.02 (and 160 vs 200 max tokens)	Production decoding differed from what was measured	Server uses 1.02 for the meeting adapter; eval uses 200 tokens. Live re-run matched offline results exactly
6	Docstrings and error messages referenced removed paths (scripts/, ../DealIntelligence_CRM, ../Llama3_CRM)	Misleading run instructions	Updated in 22 files; comments and messages only

8. Recommended next steps

#	Action	Detail	Priority
1	Resolve the XGBoost ground-truth question	Establish which labelled dataset is authoritative. If the 200-row files are genuine, retrain on the combined data, commit the training CSV, and re-run the independent re-scoring above. Target: MAE under 6 on every labelled set.	High
2	Recompute lead_score and engagement_score in the AI server	Both are pure functions of fields the deal_state adapter already gets 100% right. Computing them after generation (as the backend already does) should lift whole-state accuracy from 31% to roughly 85% without retraining.	Medium
3	Improve meeting sentiment (82%)	Add training examples of neutral meetings with mildly positive wording, retrain lead_meeting, and re-run evaluate_meeting.py. Target: sentiment $\geq 92\%$ and priority band $\geq 90\%$.	Medium
4	Protect lead-scoring post-processing	Add a test asserting the served score equals the bullet-rule total, so reconcile_output cannot be removed or bypassed without a failing build.	Medium
5	Record task-level metrics with each adapter	Write these results into weights/PROVENANCE.json for lead_scoring and lead_meeting (only deal_state has one) and re-run the three evaluations after every retrain.	Low
6	Commit the fixes	Review and commit the changes in section 7, then run the full backend test suite.	Low

Operational note. Do not run an evaluation while the AI server is serving on the same 16 GB card. During this run, two model copies generating at once coincided with an NVIDIA driver reset (nvlddmkm event 153) and the AI server process stopped. Evaluations completed reliably with the GPU to themselves.

9. Reproducing these results

From the ai/ directory, with the AI server stopped:

Adapter	Command	Time on A4000
lead_meeting	python adapters/lead_meeting/eval/evaluate_meeting.py	~6 min
lead_scoring	python adapters/lead_scoring/eval/eval_format_consistency.py --n 50	~12 min
deal_state	python adapters/deal_state/eval/evaluate.py	~25 min
deal_state end-to-end	python adapters/deal_state/eval/evaluate_end_to_end.py --limit 80	~25 min
Live stack	python tools/smoke_test.py · python tools/system_test.py (servers running)	~3 min

The raw lead-scoring figures (without reconcile_output) and the XGBoost re-scoring were produced by ad-hoc scripts that reuse the evaluation harness; they are not yet part of the repository.